

整体调用过程

1. 创建协程池，并且将协程对象放入协程池中
2. 协程对象中包含：
 1. 协程执行的函数：F f
 2. 协程函数的参数：std::tuple<Args...> args（使用tuple来进行保存）
 3. 继承 basic_context，保存协程的寄存器和栈信息
 4. 构造函数，初始化协程函数和参数，并且设置协程栈信息
 5. 开始运行协程，暂停协程，重新恢复运行等函数
3. 调用serial_execute_all()，此时会轮询协程池中的协程，如果未完成并且已经ready了，则开始执行该协程
4. 使用resume()函数，其中会调用coroutine_switch()接口开始执行汇编部分，此时会：
 1. 保存调度器协程的寄存器信息于caller_registers数组中
 2. 根据协程对象的callee_registers数组中的信息来恢复协程的寄存器状态
 - 注意，此时的callee_registers中rsp初始化成了专属栈的栈顶地址（因为专属栈的隔离，所以其实只需要将rsp指向专属栈的栈顶即可完成栈状态的恢复）
 - rip为coroutine_entry的入口地址
 - r12 寄存器为 coroutine_main 的地址
 - r13 寄存器为 coroutine_main 的参数，即当前协程对象的地址
5. 执行完切换函数后，由于rip被恢复成了coroutine_entry的入口地址，所以开始调用coroutine_entry()
6. coroutine_entry()也是一个汇编接口，会将r13中存的当前协程对象的地址传入rdi（也就是作为参数），然后调用coroutine_main,开始执行该协程
7. coroutine_main中会调用run函数，run函数会进行宏展开，然后开始执行协程的函数f
 1. 如果执行过程中调用yield(),则会调用coroutine_switch()接口，此时会将保存当前的寄存器状态到callee_registers数组中，并且通过caller_registers数组恢复调度器的寄存器状态
 - 注意切入协程和切出协程的调用参数顺序不同，切入的时候是保存到caller_registers中，切出的时候是保存到callee_registers中
 2. 如果没有调用yield(),而是执行完了整个协程，则会回到coroutine_main()中，此时会将该协程对象标记为finished，然后调用coroutine_switch()接口，将寄存器状态恢复到调度器的状态，继续循环

Task 1

填充的代码

1. common.h
yield()函数，用于实现协程的切出

```

1 void yield() {
2     if (!g_pool->is_parallel) {
3         // 从 g_pool 中获取当前协程状态
4         auto context = g_pool->coroutines[g_pool->context_id];
5         // 调用 coroutine_switch 切换到 coroutine_pool 上下文
6         // 第一个参数是保存当前状态 (callee)，第二个参数是恢复到的状态 (caller)
7         coroutine_switch(context->callee_registers, context->caller_registers);
8     }
9 }

```

2. context.h

resume()函数，用于开始协程

```

1 virtual void resume() {
2     // 调用 coroutine_switch
3     // 在汇编中保存 callee-saved 寄存器，设置协程函数栈帧，然后将 rip 恢复到协程 yield
4     // 之后所需要执行的指令地址。
5     // 第一个参数是保存当前状态 (caller)，第二个参数是恢复到的状态 (callee)
6     coroutine_switch(caller_registers, callee_registers);
7 }

```

3. coroutine.h

协程的调度执行

```

1 while(cnt > 0) {
2     for(int i = 0; i < coroutines.size(); i++) {
3         auto context = coroutines[i];
4         if(context->finished) {
5             continue;
6         }
7         //! 注意要设置当前正在执行的协程id
8         context_id = i;
9         // 使用resume，会切换到协程并且执行
10        context->resume();
11        // 如果结束了，则数量减1
12        if(context->finished) {
13            cnt--;
14        }
15    }
16 }
17 }

```

4. context.S

实现coroutine_switch()接口，用于切换协程的寄存器状态

```

1 # 保存 callee-saved 寄存器到 %rdi 指向的上下文
2 # 对于caller-saved寄存器，如果有用的话在调用协程切换函数的时候就会进行保留了
3 # 保存的上下文中 rip 指向 ret 指令的地址 (.coroutine_ret)
4 # 第一个参数中保存当前的状态
5 # 根据寄存器枚举类来确定各个寄存器的保存顺序
6 movq %rsp, 64(%rdi)
7 movq %rbx, 72(%rdi)
8 movq %rbp, 80(%rdi)

```

```
9      movq %r12, 88(%rdi)
10     movq %r13, 96(%rdi)
11     movq %r14, 104(%rdi)
12     movq %r15, 112(%rdi)
13     # rip保存为 .coroutine_ret的地址(这里是rip相对寻址)
14     leaq .coroutine_ret(%rip), %rax
15     movq %rax, 120(%rdi)
16
17
18     # 从 %rsi 指向的上下文恢复 callee-saved 寄存器
19     # 最后 jmpq 到上下文保存的 rip
20     movq 64(%rsi), %rsp
21     movq 72(%rsi), %rbx
22     movq 80(%rsi), %rbp
23     movq 88(%rsi), %r12
24     movq 96(%rsi), %r13
25     movq 104(%rsi), %r14
26     movq 112(%rsi), %r15
27     # jmp到保存的rip处(其实就是entry)
28     jmpq *120(%rsi)
```

通过测试用例，验证了协程的基本功能：

```
> ./bin/sample
```

```
in show(): 0
```

```
in show(): 0
```

```
in show(): 1
```

```
in show(): 1
```

```
in show(): 2
```

```
in show(): 2
```

```
in show(): 3
```

```
in show(): 3
```

```
in show(): 4
```

```
in show(): 4
```

```
in main(): 0
```

```
in main(): 0
```

```
in main(): 1
```

```
in main(): 1
```

```
in main(): 2
```

```
in main(): 2
```

```
in main(): 3
```

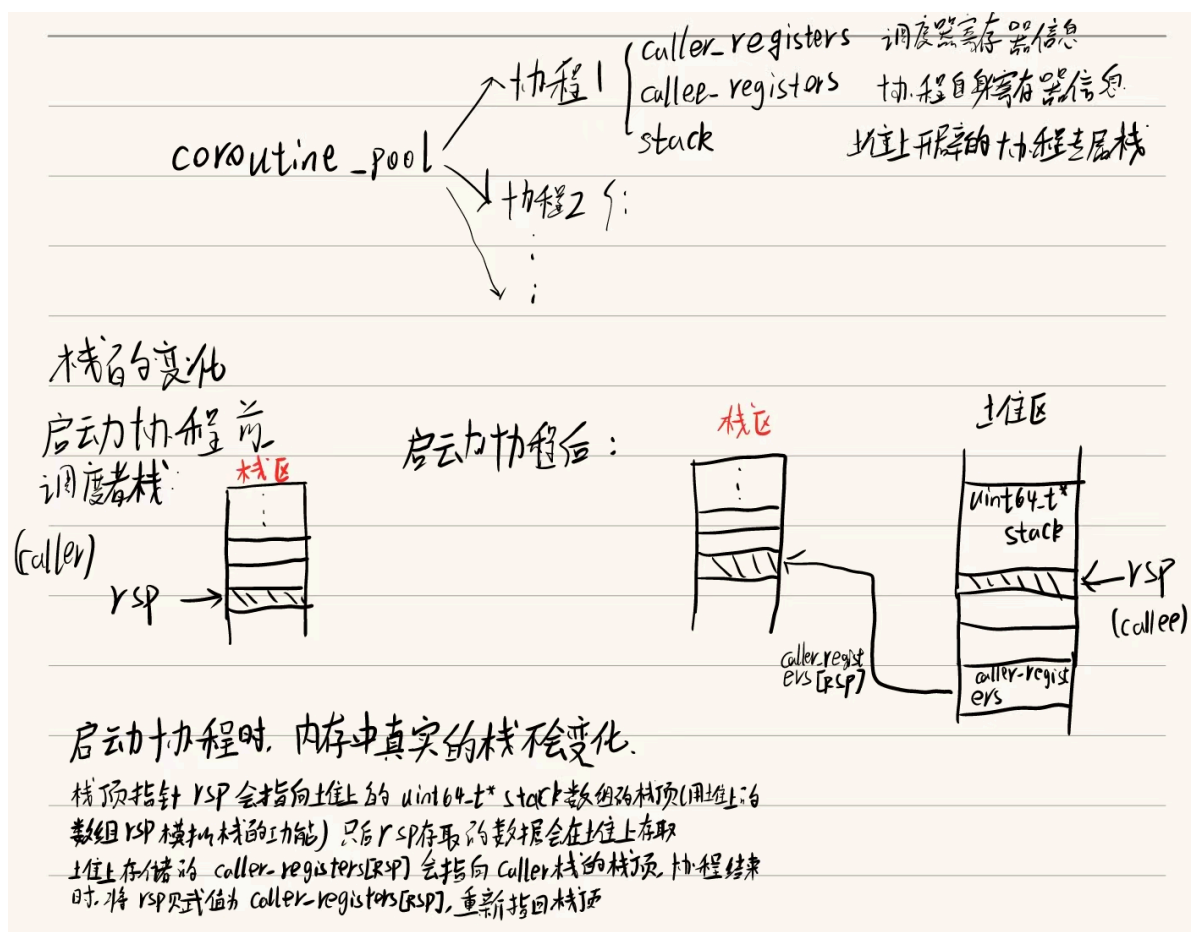
```
in main(): 3
```

```
in main(): 4
```

```
in main(): 4
```

```
~/2-autumn/i/coroutinelab master !5 ?1
```

栈变化的过程



协程如何开始执行

1. 协程的初始状态

每个协程被创建为一个结构 `coroutine_context`, 其中包含了

1. 协程执行的函数: `F f`
2. 协程函数的参数: `std::tuple<Args...> args` (使用 `tuple` 来进行保存)
3. 继承 `basic_context`, 保存协程的寄存器和栈信息
4. 构造函数, 初始化协程函数和参数, 并且设置协程栈信息
5. 开始运行协程, 暂停协程, 重新恢复运行等函数

```
1 uint64_t *stack;  
2 uint64_t stack_size;  
3 // 调度器协程的状态  
4 uint64_t caller_registers[(int)Registers::RegisterCount];  
5 // 当前协程本身的状态  
6 uint64_t callee_registers[(int)Registers::RegisterCount];  
7 F f;  
8 std::tuple<Args...> args;
```

在构造函数中, 在堆上开辟一块内存 `stack`, 用来模拟协程的栈, 并且使用变量 `rsp` 指向16位对齐的栈顶地址

协程的寄存器状态数组中, `rip` 设置为 `coroutine_entry` 的入口地址, `rsp` 设置为16位对齐的栈顶地址, `r12` 设置为 `coroutine_main` 的入口地址, `r13` 设置为当前协程对象的地址

```

1  stack = new uint64_t[stack_size];
2      // 对齐到 16 字节边界
3      uint64_t rsp = (uint64_t)&stack[stack_size - 1];
4      rsp = rsp - (rsp & 0xF);
5      void coroutine_main(struct basic_context * context);
6      callee_registers[(int)Registers::RSP] = rsp;
7      // 协程入口是 coroutine_entry
8      callee_registers[(int)Registers::RIP] = (uint64_t)coroutine_entry;
9      // 设置 r12 寄存器为 coroutine_main 的地址
10     callee_registers[(int)Registers::R12] = (uint64_t)coroutine_main;
11     // 设置 r13 寄存器, 用于 coroutine_main 的参数
12     callee_registers[(int)Registers::R13] = (uint64_t)this;

```

2. context_switch()

serial_execute_all()中启动某个协程, 此时会调用resume函数, 启动context_switch接口, 通过汇编代码保存当前调度器协程的状态, 然后将当前的寄存器状态恢复到协程的状态, 继续执行协程函数

```

1  void resume() {
2      // 调用 coroutine_switch
3      // 在汇编中保存 callee-saved 寄存器, 设置协程函数栈帧, 然后将 rip 恢复到协程
yield 之后所需要执行的指令地址。
4      // 第一个参数是保存当前状态 (caller), 第二个参数是恢复到的状态 (callee)
5      coroutine_switch(caller_registers, callee_registers);
6  }

```

- 保存当前状态: 将当前的寄存器状态保存到caller_registers数组中

```

1  movq %rsp, 64(%rdi)
2  movq %rbx, 72(%rdi)
3  movq %rbp, 80(%rdi)
4  movq %r12, 88(%rdi)
5  movq %r13, 96(%rdi)
6  movq %r14, 104(%rdi)
7  movq %r15, 112(%rdi)
8  # rip保存为 .coroutine_ret的地址(这里是rip相对寻址)
9  leaq .coroutine_ret(%rip), %rax
10 movq %rax, 120(%rdi)

```

- 恢复状态: 将协程的寄存器状态从callee_registers数组中恢复到当前寄存器中

```

1  # 从 %rsi 指向的上下文恢复 callee-saved 寄存器
2  # 最后 jmpq 到上下文保存的 rip
3  movq 64(%rsi), %rsp
4  movq 72(%rsi), %rbx
5  movq 80(%rsi), %rbp
6  movq 88(%rsi), %r12
7  movq 96(%rsi), %r13
8  movq 104(%rsi), %r14
9  movq 112(%rsi), %r15
10 # jmp到保存的rip处 (其实就是entry)
11 jmpq *120(%rsi)

```

执行完状态切换之后, 此时的寄存器状态如下:

- %rsp指向协程的专属栈stack的栈顶
- %rip中存放了coroutine_entry的入口地址
- %r12中存放了coroutine_main的入口地址
- %r13中存放了当前协程对象的地址

3. coroutine_entry()

完成context_switch后，下一步会跳转到coroutine_entry的入口地址，在coroutine_entry中，会调用coroutine_main函数，开始执行协程函数

```
1 .coroutine_entry:
2   # 传递协程对象指针作为参数
3   movq %r13, %rdi
4   # 调用 coroutine_main
5   callq *%r12
```

4. coroutine_main()

coroutine_main()是协程的统一入口函数，所有的协程函数都必须通过coroutine_main来执行

```
1 void coroutine_main(struct basic_context * context) {
2     // 多态调用，开始运行协程
3     context->run();
4     context->finished = true;
5     coroutine_switch(context->callee_registers, context->
6 >caller_registers);
7     // unreachable
8     assert(false);
9 }
```

其中会调用协程的run()函数，run函数中会通过宏定义展开来调用协程函数

```
1 virtual void run() { CALL(f, args); }
```

如果函数正常结束，则标记此次协程完成，并调用 coroutine_switch(callee_registers, caller_registers)切换回原本的调度器协程。

如果协程执行的函数里有 yield 函数，那么 yield 函数会主动调用 coroutine_switch(context->callee_registers, context->caller_registers) 将协程停止并挂起，然后恢复调用此协程的caller函数的运行状态。

Task 2

实现原理

1. 当协程调用sleep(ms)的时候，会记录当前的时间并且设置一个超时检测函数
 2. 将当前协程标记为未就绪状态，并且让出控制权
 3. 调度器在轮询的时候会调用每个未就绪函数的ready_func，检测是否已经超过了指定的睡眠时间
 4. 如果超过了，则将协程标记为就绪并且执行，如果没有，继续轮询
- 精度问题
会出现轮询延迟，协程调用sleep后会立刻让出控制权，但是只有当下一次轮询到他的时候才会去检查是否超时
如果其他协程的执行时间较长，可能会导致sleep的协程长时间得不到检查

所以一个协程sleep的时间实际上会比规定的时间长一些，要加上等待再次被轮询到的时间

在睡眠排序中，每个协程只会调用sleep和printf函数，执行时间较短，所以可以忽略轮询延迟的问题，认为协程同时开始并且结束的时间决定为sleep的时间

填充的代码

1. coroutine_pool.h

填充serial_execute_all()函数,考虑ready状态，如果不是就绪状态要调用ready_func检查是否已经超时，如果超时了则设置为就绪状态并且执行，如果没有超时则继续轮询

```
1 // 总的协程数
2 int cnt = coroutines.size();
3 // 只要还有协程，就要一直轮询执行
4 while(cnt > 0) {
5     for(int i = 0; i < coroutines.size(); i++) {
6         auto context = coroutines[i];
7         // 如果已经结束了，则退出
8         if(context->finished) {
9             continue;
10        }
11        // 如果不是就绪状态，检查是否超时
12        // 如果超时了，则设置为就绪状态，否则continue
13        if(!context->ready) {
14            if(context->ready_func()) {
15                context->ready = true;
16            }
17            else {
18                continue;
19            }
20        }
21        //经过前面的过滤，此处可以确保当前的协程未结束并且就绪
22        //! 注意要设置当前正在执行的协程id
23        context_id = i;
24        // 使用resume，会切换到协程并且执行
25        context->resume();
26        // 如果结束了，则数量减1
27        if(context->finished) {
28            cnt--;
29        }
30    }
31 }
32 }
```

2. common.h

填充sleep(ms)函数，记录当前时间并且设置一个超时检测函数，将当前协程标记为未就绪状态，并且让出控制权

```
1 else {
2     // 从 g_pool 中获取当前协程状态
3     // 获取当前协程
4     auto context = g_pool->coroutines[g_pool->context_id];
5
6     // 获取当前时间，更新 ready_func
7     // ready_func: 检查当前时间，如果已经超时，则返回 true
```



```

8      auto cur = get_time();
9      // 使用lambda表达式注册ready_func
10     context->ready_func = [cur, ms]() -> bool {
11         // 如果当前时间减去开始时间大于等于规定的时间，就返回 true
12         return std::chrono::duration_cast<std::chrono::milliseconds>(get_time()
- cur)
13             .count() >= ms;
14     };
15     // 设置状态为未就绪
16     context->ready = false;
17     // 调用 coroutine_switch 切换到 coroutine_pool 上下文
18     coroutine_switch(context->callee_registers, context->caller_registers);
19 }

```

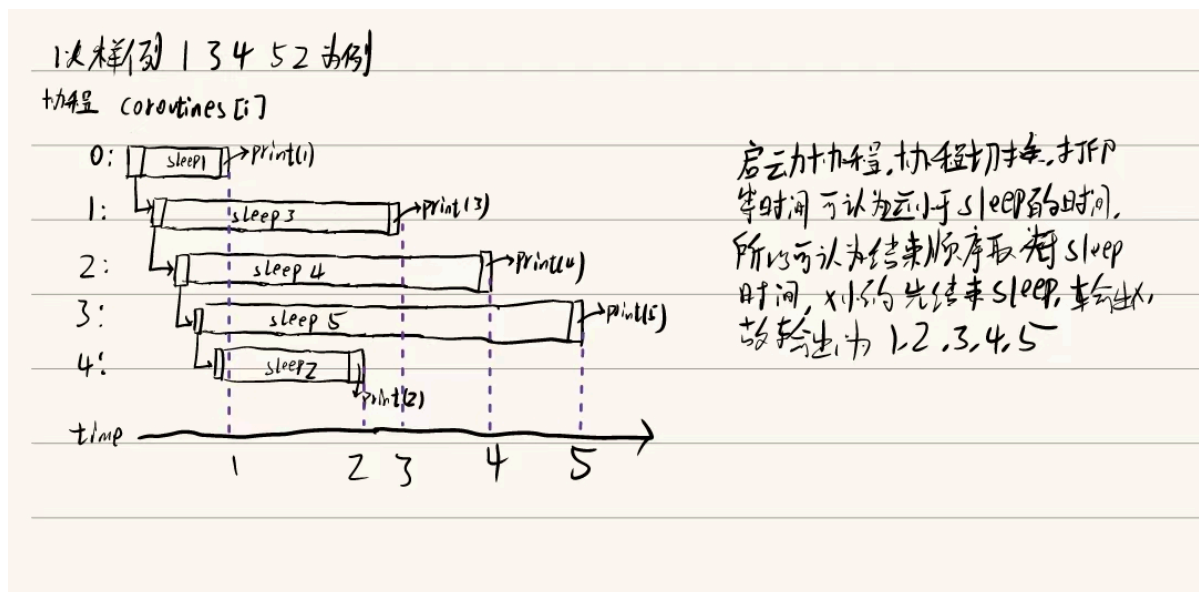
验证结果

```

● > ./bin/sleep_sort
5
1 3 4 5 2
1
2
3
4
5
~/2-autumn/i/coroutinelab master !6 ?3

```

协程运行情况



优化方法

- 问题：每次都要轮询所有协程的状态，导致额外的开销

可以引入优先级的思想，优先级也就是醒来的时间，时间越短的协程优先级越高，先被轮询到。在进程池中维护一个优先队列，每次有进程调用sleep的时候，直接计算出进程醒来的时间，并且将进程插入到优先队列中（以醒来的时间为优先级，时间越早优先级越高）

则调度器进行调度的时候可以利用优先队列，先去检查是否有进程超时，如果有超时的进程，则将其设置为就绪状态，并且执行。

如果没有超时的进程，则继续轮询其他协程。

调度过程示例代码：

```
1 while (true) {
2     // 1. 检查睡眠队列中是否有协程需要唤醒
3     while (!scheduler->sleeping_queue.empty()) {
4         PriorityQueueElement front = scheduler->sleeping_queue.top();
5         // 如果队首协程还未到时间，则退出
6         if (front.wakeup_time > current_time) {
7             break;
8         }
9         // 唤醒协程，从睡眠队列移除
10        scheduler->sleeping_queue.pop();
11        front.co->status = READY;
12        // 添加到就绪队列
13        scheduler->ready_queue.push(front.co);
14    }
15    // 2. 从就绪队列选择一个协程执行
16    if (!scheduler->ready_queue.empty()) {
17        Coroutine* next = scheduler->ready_queue.pop();
18        scheduler->current = next;
19        next->status = RUNNING;
20        // 切换到该协程执行
21        switchto(scheduler->main, next);
22        // 协程执行完毕后返回
23        if (next->status == DEAD) {
24            // 清理已终止的协程资源
25            cleanupCoroutine(next);
26        }
27    } else if (!scheduler->sleeping_queue.empty()) {
28        // 如果没有就绪协程但有睡眠协程，可以等待到最近唤醒时间
29        PriorityQueueElement front = scheduler->sleeping_queue.top();
30        int64_t current_time = getCurrentTimeMillis();
31        int64_t wait_time = front.wakeup_time - current_time;
32    }
33 }
```

分析：

原本的轮询平均需要 $O(n)$ 的时间才能启动一个协程，引入了优先队列之后，虽然每次sleep需要花费 $O(\log n)$ 的时间来插入，但是每次检查可以直接检查优先队列的队头元素，只需要 $O(1)$ 。

Task 3

原理

二分查找的局部性比较差，所以缓存机制容易失效，当缓存失效的时候cpu需要从内存中加载数据。如果使用原本的二分查找，则加载数据的时候是阻塞整个进程的，导致查找效率降低

而使用协程优化二分查找，可以在缓存失效的时候切换到其他协程进行执行，cpu异步的从内存中调用数据。等到轮询到使用存取指令的协程的时候，cpu已经将需要的数据调取到了缓存中，可以直接使用，这样可以避免阻塞整个进程，提高查找效率。

填充的代码

binary_search.h

```
1      //! __builtin_prefetch: GCC 提供的一个内置函数，用于在数据被实际访问之前，将其提前加载到 CPU 缓存中，从而减少内存访问延迟并提高程序性能。
2      //! 参数为地址
3      // 这里调用预取后就不继续执行了，而是退出该协程，先执行其他的协程
4      // 等其他的协程执行完后，再回来执行这里
5      __builtin_prefetch(&table[probe]);
6      yield();
```

性能分析

对于l和m的测试在我的本地电脑中的wsl测试，对于b的测试在课程服务器进行

l对性能的影响

1. l = 32, m = 1000000, b = 16

```
1 > ./bin/binary_search
2 Size: 4294967296
3 Loops: 1000000
4 Batch size: 16
5 Initialization done
6 naive: 1916.28 ns per search, 59.88 ns per access
7 coroutine batched: 1798.87 ns per search, 56.21 ns per access
```

2. l = 31, m = 1000000, b = 16

```
1 > ./bin/binary_search -l 31
2 Size: 2147483648
3 Loops: 1000000
4 Batch size: 16
5 Initialization done
6 naive: 1573.48 ns per search, 50.76 ns per access
7 coroutine batched: 1148.15 ns per search, 37.04 ns per access
```

3. l = 30, m = 1000000, b = 16

```
1 > ./bin/binary_search -l 30
2 Size: 1073741824
3 Loops: 1000000
4 Batch size: 16
5 Initialization done
6 naive: 1243.18 ns per search, 41.44 ns per access
7 coroutine batched: 906.62 ns per search, 30.22 ns per access
```

4. l = 29, m = 1000000, b = 16

```
1 > ./bin/binary_search -l 29
2 Size: 536870912
3 Loops: 1000000
4 Batch size: 16
5 Initialization done
6 naive: 1083.21 ns per search, 37.35 ns per access
7 coroutine batched: 862.71 ns per search, 29.75 ns per access
```

5. l = 28, m = 1000000, b = 16

```
1 > ./bin/binary_search -l 28
2 Size: 268435456
3 Loops: 1000000
4 Batch size: 16
5 Initialization done
6 naive: 856.78 ns per search, 30.60 ns per access
7 coroutine batched: 719.79 ns per search, 25.71 ns per access
```

6. l = 27, m = 1000000, b = 16

```
1 > ./bin/binary_search -l 27
2 Size: 134217728
3 Loops: 1000000
4 Batch size: 16
5 Initialization done
6 naive: 700.42 ns per search, 25.94 ns per access
7 coroutine batched: 646.79 ns per search, 23.96 ns per access
```

7. l = 26, m = 1000000, b = 16

```
1 > ./bin/binary_search -l 26
2 Size: 67108864
3 Loops: 1000000
4 Batch size: 16
5 Initialization done
6 naive: 458.00 ns per search, 17.62 ns per access
7 coroutine batched: 570.04 ns per search, 21.92 ns per access
```

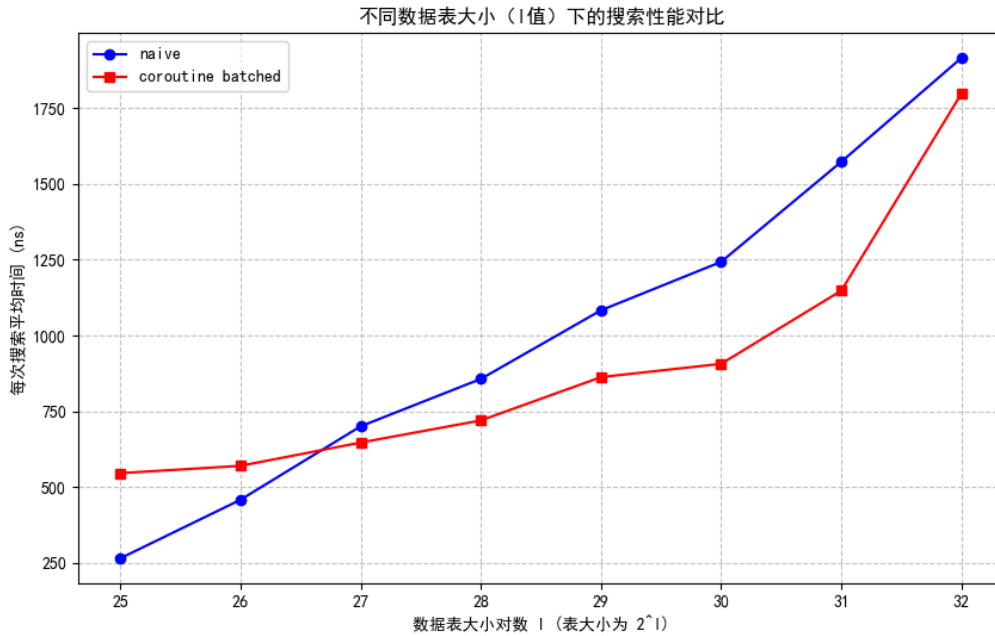
8. l = 25, m = 1000000, b = 16

```

1 > ./bin/binary_search -l 25
2 Size: 33554432
3 Loops: 1000000
4 Batch size: 16
5 Initialization done
6 naive: 264.83 ns per search, 10.59 ns per access
7 coroutine batched: 545.80 ns per search, 21.83 ns per access

```

分析图像如下：



可以发现n比较大的时候，协程的性能优势更加明显，当n小于27的时候，普通的二分查找的性能要优于协程的二分查找

原因分析：当数组长度较小的时候，二分查找缓存更容易命中，而协程优化的版本需要频繁的进行协程的切换，协程切换会带来额外的开销，当n较小的时候对于缓存的优化效果无法抵消协程切换的开销，所以在n较小的时候，普通的二分查找的性能要优于协程的二分查找

改变m

理论上，我们测得的是平均的时间，次数对平均影响应该没有大的影响

```

1 > ./bin/binary_search -l 31 -m 3000000
2 Size: 2147483648
3 Loops: 3000000
4 Batch size: 16
5 Initialization done
6 naive: 1509.51 ns per search, 48.69 ns per access
7 coroutine batched: 1143.79 ns per search, 36.90 ns per access

```

```

1 > ./bin/binary_search -l 31 -m 2000000
2 Size: 2147483648
3 Loops: 2000000
4 Batch size: 16
5 Initialization done
6 naive: 1595.32 ns per search, 51.46 ns per access
7 coroutine batched: 1360.87 ns per search, 43.90 ns per access

```

```
1 > ./bin/binary_search -l 31 -m 100000
2 Size: 2147483648
3 Loops: 100000
4 Batch size: 16
5 Initialization done
6 naive: 1605.12 ns per search, 51.78 ns per access
7 coroutine batched: 1400.96 ns per search, 45.19 ns per access
```

```
1 > ./bin/binary_search -l 31 -m 750000
2 Size: 2147483648
3 Loops: 750000
4 Batch size: 16
5 Initialization done
6 naive: 1535.54 ns per search, 49.53 ns per access
7 coroutine batched: 1304.41 ns per search, 42.08 ns per access
```

固定l=31,改变m可以发现协程版本的还是优于普通的二分查找的, 变化较小

改变b

1. -l 31 -m 1000000 -b 2
Size: 2147483648
Loops: 1000000
Batch size: 2
Initialization done
naive: 1139.52 ns per search, 36.76 ns per access
coroutine batched: 1564.78 ns per search, 50.48 ns per access
2. -l 31 -m 999999 -b 3
Size: 2147483648
Loops: 999999
Batch size: 3
Initialization done
naive: 1078.92 ns per search, 34.80 ns per access
coroutine batched: 1280.65 ns per search, 41.31 ns per access
3. -l 31 -m 1000000 -b 4
Size: 2147483648
Loops: 1000000
Batch size: 4
Initialization done
naive: 1101.96 ns per search, 35.55 ns per access
coroutine batched: 1074.24 ns per search, 34.65 ns per access
4. -l 31 -m 1000000 -b 6
Size: 2147483648
Loops: 1000000
Batch size: 6
Initialization done
naive: 1133.27 ns per search, 36.56 ns per access
coroutine batched: 1066.18 ns per search, 34.39 ns per access

5. -l 31 -m 1000002 -b 8
Size: 2147483648
Loops: 1000000
Batch size: 8
Initialization done
naive: 1111.98 ns per search, 35.87 ns per access
coroutine batched: 1058.17 ns per search, 34.13 ns per access
6. -l 31 -m 1000002 -b 10
Size: 2147483648
Loops: 1000000
Batch size: 10
Initialization done
naive: 1259.86 ns per search, 40.64 ns per access
coroutine batched: 1075.88 ns per search, 34.71 ns per access
7. -l 31 -m 1000002 -b 12
Size: 2147483648
Loops: 1000008
Batch size: 12
Initialization done
naive: 1269.12 ns per search, 40.94 ns per access
coroutine batched: 1066.82 ns per search, 34.41 ns per access
8. -l 31 -m 1000002 -b 14
Size: 2147483648
Loops: 1000006
Batch size: 14
Initialization done
naive: 1222.42 ns per search, 39.43 ns per access
coroutine batched: 1055.84 ns per search, 34.06 ns per access
9. -l 31 -m 1000000 -b 20
Size: 2147483648
Loops: 1000000
Batch size: 20
Initialization done
naive: 1263.94 ns per search, 40.77 ns per access
coroutine batched: 1090.26 ns per search, 35.17 ns per access
10. -l 31 -m 1000008 -b 24
Size: 2147483648
Loops: 1000008
Batch size: 24
Initialization done
naive: 1256.79 ns per search, 40.54 ns per access
coroutine batched: 1085.42 ns per search, 35.01 ns per access
11. -l 31 -m 1000000 -b 25
Size: 2147483648
Loops: 1000000
Batch size: 25
Initialization done

naive: 1254.39 ns per search, 40.46 ns per access
coroutine batched: 1084.75 ns per search, 34.99 ns per access

12. -l 31 -m 1000000 -b 26

Size: 2147483648

Loops: 1000012

Batch size: 26

Initialization done

naive: 1257.96 ns per search, 40.58 ns per access

coroutine batched: 1387.68 ns per search, 44.76 ns per access

13. -l 31 -m 1000000 -b 28

Size: 2147483648

Loops: 1000020

Batch size: 28

Initialization done

naive: 1273.15 ns per search, 41.07 ns per access

coroutine batched: 1562.37 ns per search, 50.40 ns per access

14. -l 31 -m 1000000 -b 30

Loops: 999990

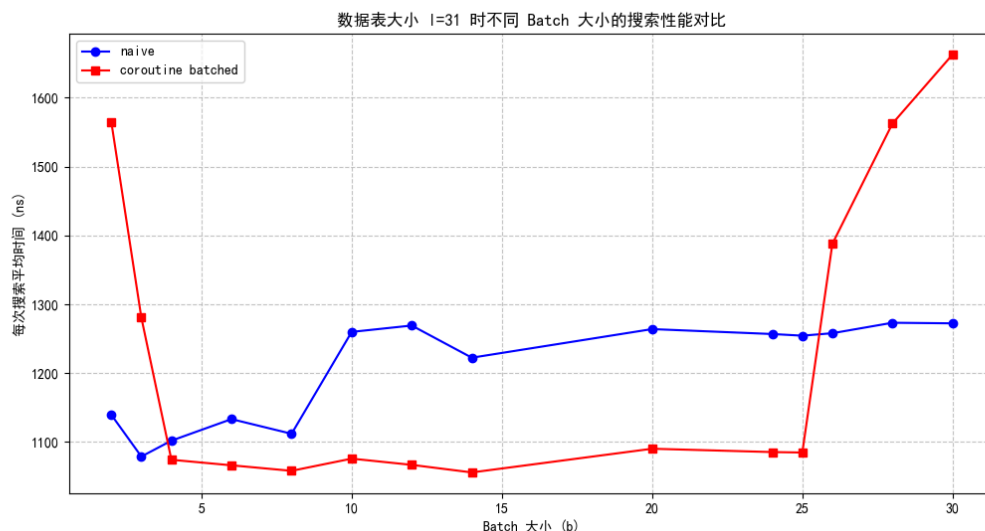
Batch size: 30

Initialization done

naive: 1272.31 ns per search, 41.04 ns per access

coroutine batched: 1662.57 ns per search, 53.63 ns per access

绘制折线图：



可以发现随着b的增加，协程版本的二分查找的效率呈现先上升，后下降的趋势，当 $b < 4$ 是，协程版本的二分查找的效率要低于普通的二分查找； $b \geq 4$ 且 $b < 26$ 时，协程版本二分查找的效率高于普通二分查找；而当 $b \geq 26$ 时，协程版本的二分查找的效率要低于普通的二分查找

当 $b < 26$ 的时候，协程数量的增加对于效率是有优化效果的，而当协程数过大时，反而有负优化

原因分析：

理想状态下，协程切换进行一轮的时间应该接近cpu存取数据的时间，这样进行一轮协程切换刚好完成数据读取

当b较小时，协程池中协程的数量较小，协程切换的开销大于其并行带来的收益

后面b增大，协程增多，并行的收益增大，因此协程版本的二分查找的效率会有提升，高于普通版本

而当b过大时，协程切换的时间已经足以覆盖cpu获取数据的时间，反而可能因为协程过多，协程切换耗费了过多的时间，效率反而降低

交流与参考

<https://stackoverflow.com/questions/57212012/how-to-load-address-of-function-or-label-into-register>

https://www.bilibili.com/video/BV12zUaBdEkV/?spm_id_from=333.1387.homepage.video_card.click&vd_source=44dca6a937737d01b3024f3f47ff8e23

总结与感想

这次实验是我第一次直接编写汇编代码进行系统调用。经过本次实验，我对程序的运行过程和本质有了更深的理解，认识到寄存器和栈的状态就可以记录一个函数的运行状态。本次实验也让我认识到了学习计算机系统的必要性。虽然现在的高级语言功能非常强大，但是汇编语言能够直接对计算机底层进行操作，其功能是高级语言无法替代的。